

Java Concurrency Impact on Singleton and Producer-Consumer Patterns

Azat Satklyčov

azats@seznam.cz,

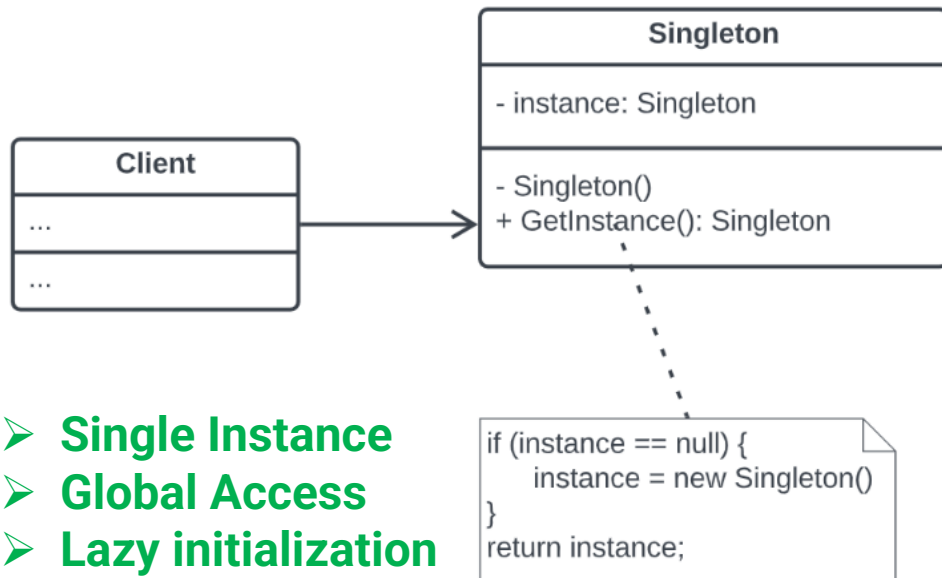
<http://sahet.net>,

<https://github.com/asatklichov/multithreading-java-vs-nodejs>

<https://medium.com/modern-mainframe/multithreading-in-java-vs-node-js-c558d59050c9>

Singleton Pattern

Singleton is a creational design pattern that ensures only **a single instance of a class** exists throughout an application and provides access to that instance from anywhere in the codebase. It is useful in situations where multiple instances of a class are undesirable or a single point of control or coordination is required. Typical examples include logging systems, database connection managers, caches, and thread pools, among others.



- **Single Instance**
- **Global Access**
- **Lazy initialization**

```
public class Singleton {  
  
    private static Singleton instance;  
  
    private final Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Is it really a Singleton? Single instance?

How can I run my slow operations faster?

Suppose, we have a task:

- Logistic company wants to generate quotation-report
[download rate – T1, process – T2, generate report – T3, ...)
- Or other example, bank wants to get clearance-report [1?, 2?, 3?, ..]



Slow operations performance is improved by using one of below mechanisms depending on use-case.

Synchronous

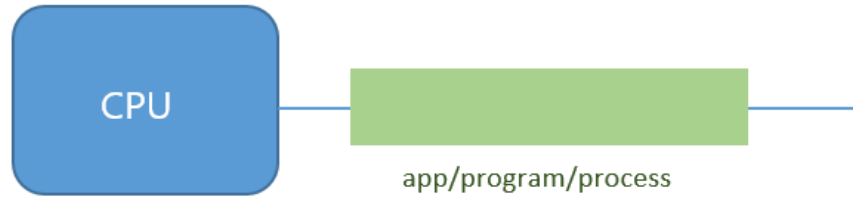
Fork new process from OS

Threads

Event Driven

Multitasking

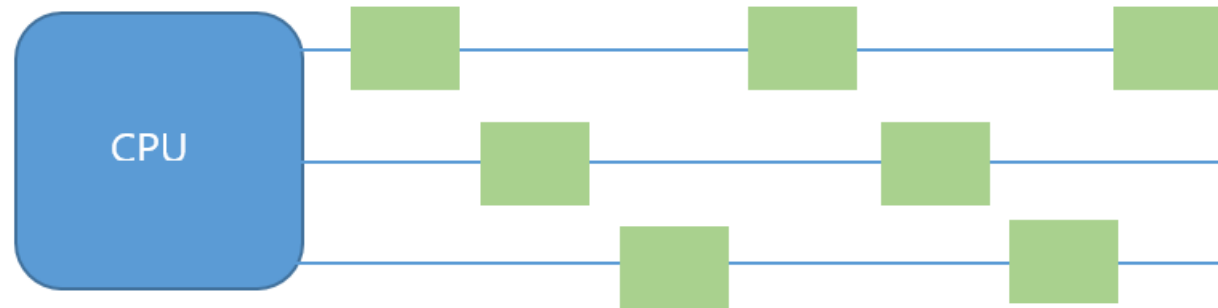
 Process



Multitasking is the concurrent execution of multiple tasks over a certain period of time. It's Processor based and thread based.

Single CPU can run single program at a time.

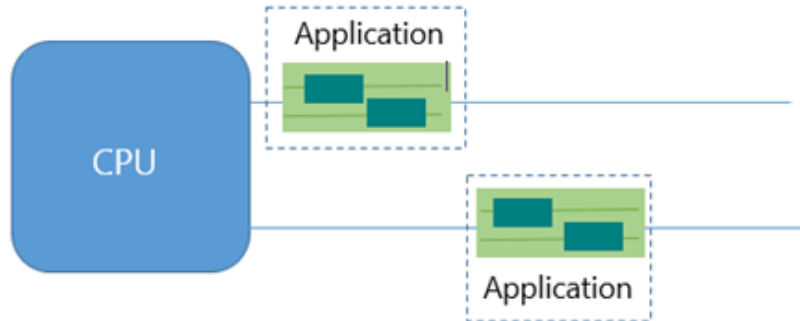
A process (program or application) has self contained execution environment.



Single CPU can run multiple program by switching between execution

Multithreading

Multiple threads executed on **single CPU**

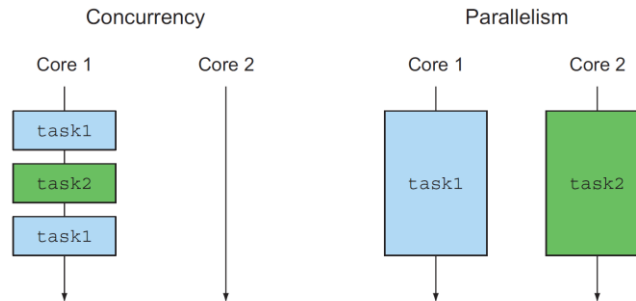


Multithreading Benefits

- Better CPU Utilization
- More responsive programs

Multithreading Costs

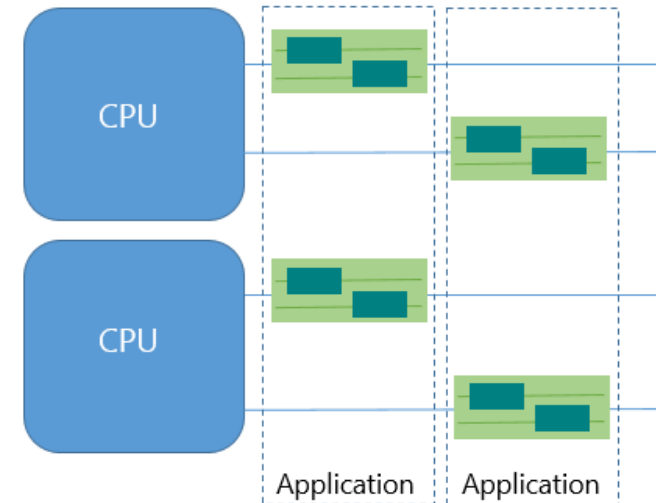
- Context switching overhead
- Increased resource consumption
- Complex design



In **concurrent programming**, there are two basic units of execution: **processes and threads** (*lightweight processes*) both provide an execution environment.

A **Thread** is a set of instructions and defined at **OS level**.

Multithreading is a type of execution model where multiple threads can run independently in a process.



Multiple threads executed on **multiple CPU**, or CPU with multiple COREs

Concurrency vs Parallelism
(runs at the same paradigm)

How can I run my slow operations faster?

Which solution is faster?

1-way: (easiest): Execute synchronously



2-way: Multithreading execution

2a) thread per task, at least three cores CPU



2b) Multithreading execution (on single core CPU, *preemptive switching by OS*)



Concurrency is when two or more tasks can be processed **at the same time** (together, concurrently) in overlapping time periods using **preemptive switching**

3-way: Asynchronous (from outside looks like **2b** but works differently inside, cooperative scheduling happens)



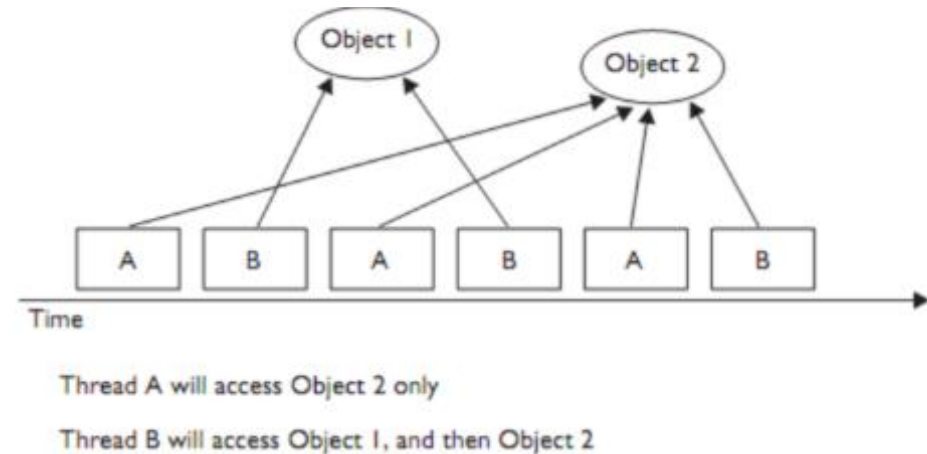
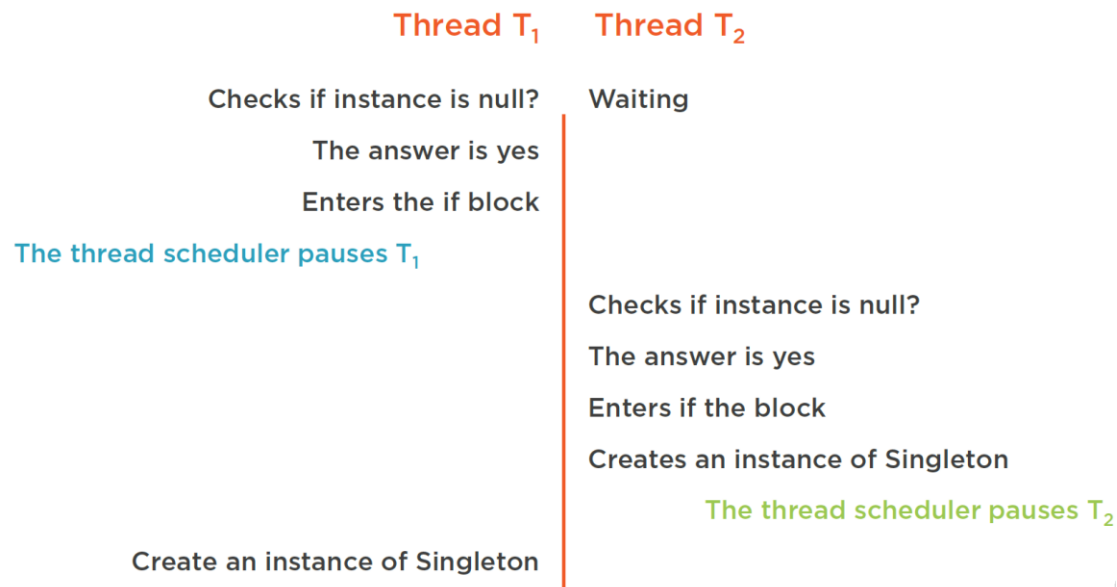
Asynchronous **at the same time** (together, concurrently) in overlapping time periods using context switching
(**non preemptive switching**)

On multicores, things happens at real at the same time. Means for non-blocking operations tasks run using multiple threads. E.g. libuv via EventLoop, or Java non-dependant (non-blocking) tasks via CompletableFuture (uses ForkJoin internally) or Reactive API

Who is responsible for this CPU time sharing (context switching)?

Race Condition (or Thread Interference). Why Mutex?

Two kinds of errors possible once threads communicate via sharing access to fields: **thread interference (race condition)** and **memory consistency errors (visibility)**. **Race condition happens** once two or more threads trying to read/write same data (variable) at the same time. **Memory consistency errors** occur when different threads have inconsistent views of what should be the same data (see next slides – for visibility). Note that: “**same time**” does not mean same thing on single and multicore CPUs.



Two race-condition patterns

- Read-modify-write
- Check-then-act

How to Prevent Race Condition? Why Mutex?

A **mutex (or mutual exclusion)** is the simplest type of **synchronizer** [*synchronized, ReentrantLock, Semaphore, Guava's monitor*] – it **ensures that only one thread can execute the critical section of a computer program at a time**. Using **synchronization** (protects - only allows one thread to execute code at a time) on block of code or on method level. However, synchronization can introduce **thread contention** (see: **Starvation, Liveness**)

Thread Unsafe Singleton Pattern

JVM creates the unique instance of this class when the class is loaded



```
public class Singleton {  
  
    private static Singleton instance;  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Single-Threaded Singleton (for Single-Threaded Model, e.g. in Java servlet)

- Non thread safe for multi-threaded app.
- Race condition happens
- No Happens Before Link
- Difficult to unit test
- **Via reflection still you** can create multiple instance
- Eagerly instantiated, slow performance
- **Serialization breaks Singleton**

See Demo.java for Singleton Race condition

- Ⓢ AThreadUnsafeSingleton 4/14/2024 7:33 PM, 1.67 kB 27 minutes ago
- Ⓢ BThreadSafeSingleton.java 4/14/2024 6:59 PM, 7.51 kB 21 minutes ago
- Ⓢ Demo.java 4/14/2024 8:14 PM, 2.69 kB Moments ago
- Ⓢ UnsafeSingleton
- Ⓢ SingletonSerializable 4/14/2024 6:57 PM, 2.35 kB 31 minutes ago
- Ⓢ SingletonSerializableEnum 4/14/2024 6:22 PM, 1.75 kB 17 minutes ago
- Ⓢ SingletonWithReflection.java 4/14/2024 7:01 PM, 3.49 kB 17 minutes ago
 - Ⓢ SingletonEnumWithReflection
- Ⓢ SingletonWithReflection

resources

re 3/26/2024 1:29 PM, 527 B

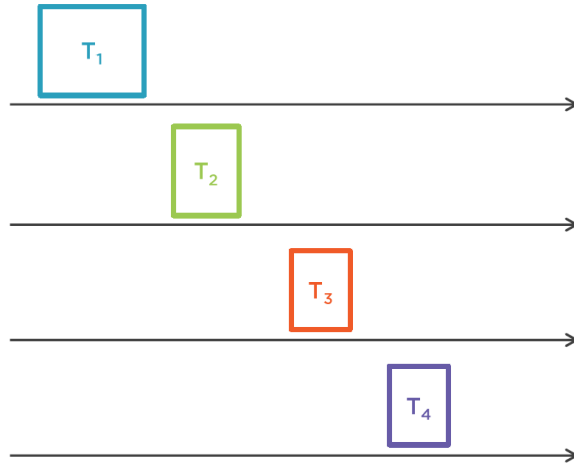
UnsafeSingleton x

```
C:\apps\Java\java-se-8u401\bin\java.exe ...  
Summary :: PROCESSOR_COUNT 16, Running Test with 16 of Threads.  
Singleton Pattern is broken after 60 try.  
Number of instances count is 9.  
Test duration 119ms  
Process finished with exit code 0
```

```
19  */  
20  private UnsafeSingleton() {  
21  }  
22  
23  /**  
24   * return the instance only if it does not exist  
25   */  
26  public static UnsafeSingleton getInstance() {  
27      if (instance == null) {  
28          instancesCount++;  
29          instance = new UnsafeSingleton();  
30      }  
31      return instance;  
32  }  
33  
34
```

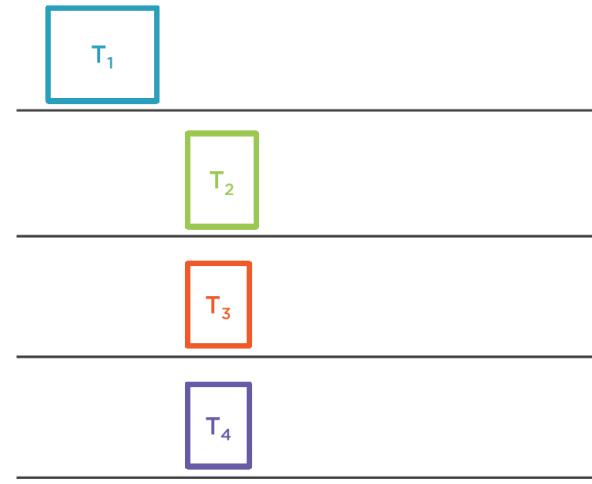
Thread Safe Singleton Pattern - Synchronized Pattern

```
public class Singleton {  
    private static Singleton instance;  
    private Singleton() {}  
    public static synchronized Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```



How it works in reality, on multicore

```
/**  
 * Use synchronized pattern, solves above problems  
 BUT brings performance Penalty  
 • On Single Core all works AS IS, threads will  
   wait the lock until released and continue in  
   this way  
 • On multicore, we have a possibility of making  
   READ operation PARALLEL but it is not possible  
 */
```



This would be ideal, so threads can READ variable in parallel

Memory Consistency Errors - Visibility

What Does Synchronization Do?

- Protects a block of code
- Guarantees this code is executed by one thread at a time
- Prevents race conditions

```
public void produce() {  
    synchronized (monitor) {  
        while (isFull(buffer)) {  
        }  
        buffer[count++] = 1;  
    }  
}
```

```
public void consume() {  
    synchronized (monitor) {  
        while (isEmpty(buffer)) {  
        }  
        buffer[--count] = 0;  
    }  
}
```

Consumer/Consumer count

1. Reads from memory
2. **Decrement/Increment** it
3. Writes count back to memory

Memory consistency errors occur when different threads have inconsistent views of what should be the same data.

- Closely bound to How multicore CPUs work
- What is Happens Before Link? Ordering Read and Write Operations on a Multicore CPU
- What does volatile mean?
- Impact of false sharing (side effect of CPU cores, caches how organized)

Computer and JAVA Memory Architecture

Memory Access Impact Visibility

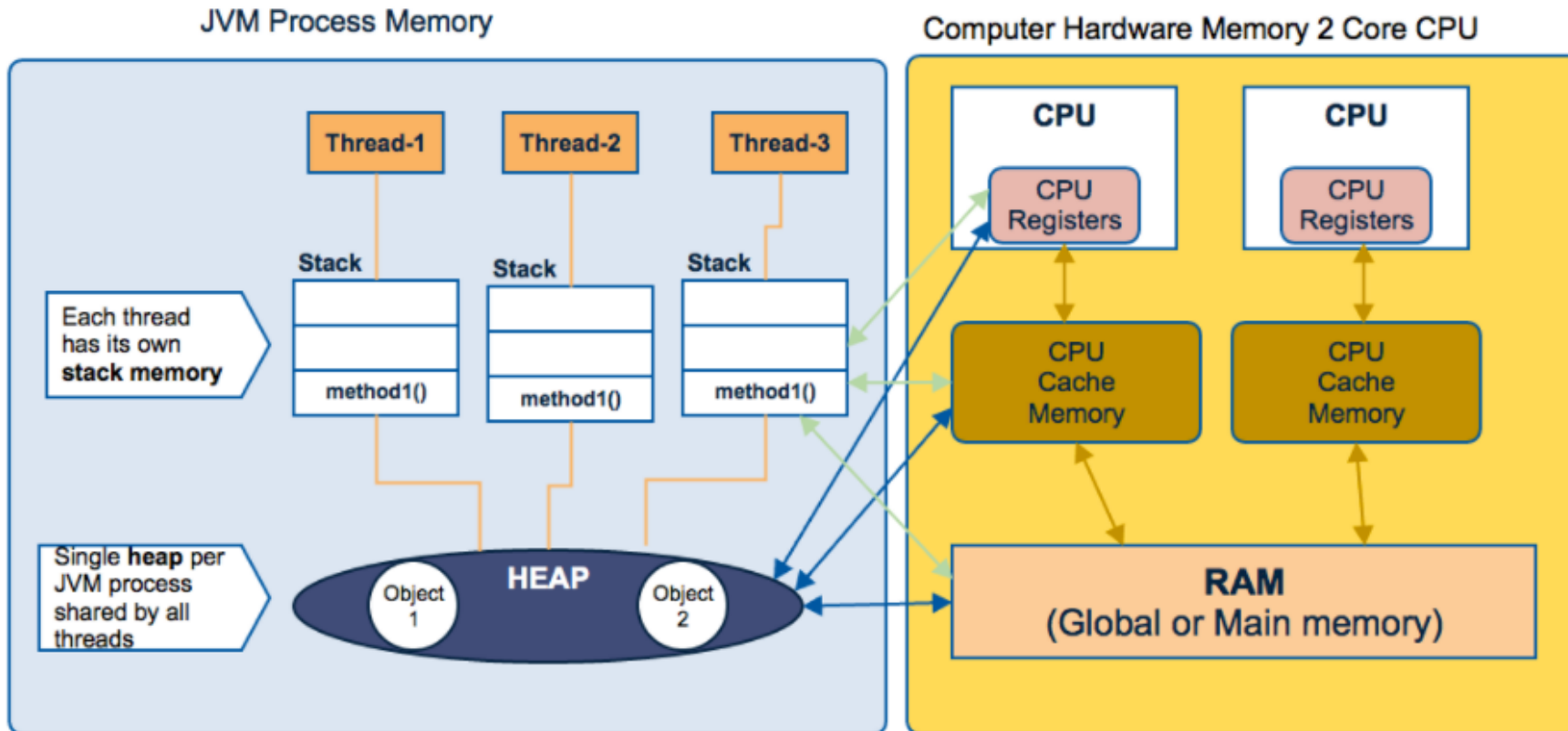
- Code works correctly when CPU **had no cache** (old computers)
- Today CPU does **not read a variable from the main memory, but from a cache**

Why it is like this?

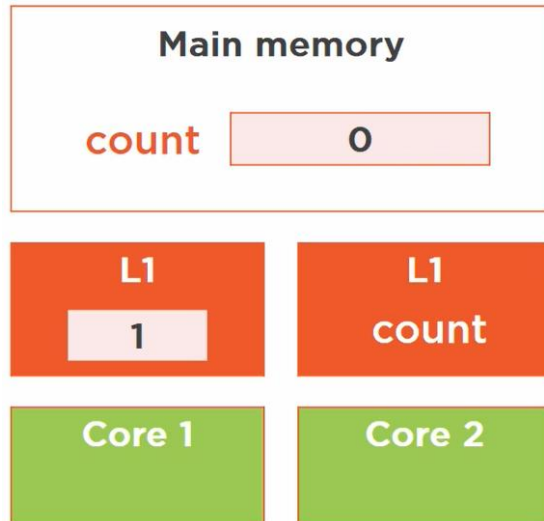
- Access to cache is much more faster than accessing Main Memory
- Access L3 is faster than to access to main memory
- Access to L2 is faster than L3
- Access to L1 is faster than L2
- Access to registers is the fastest

Drawbacks:

- Main memory size: Several GB
- L2 cache is 256 KB
- L1 cache is 32 KB



CPU Architecture



Core 1 needs count

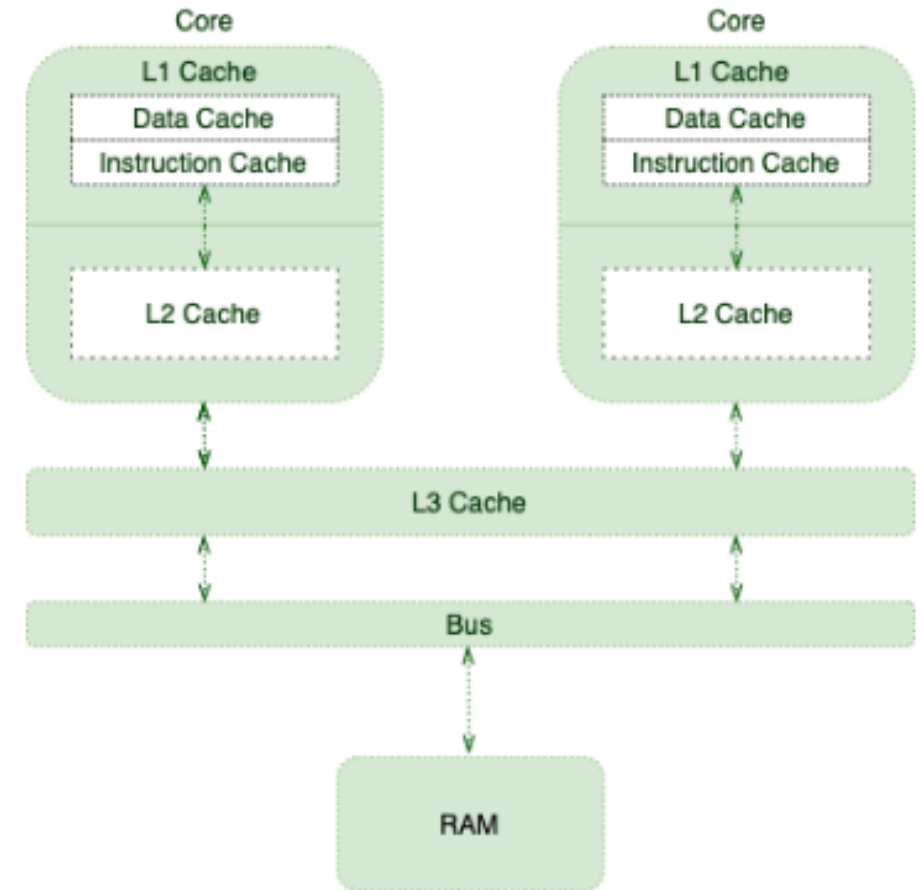
1) The variable is copied in L1

2) Core 1 can modify it

3) Core 2 also needs count

4) It should get the value 1, not 0

This is visibility!



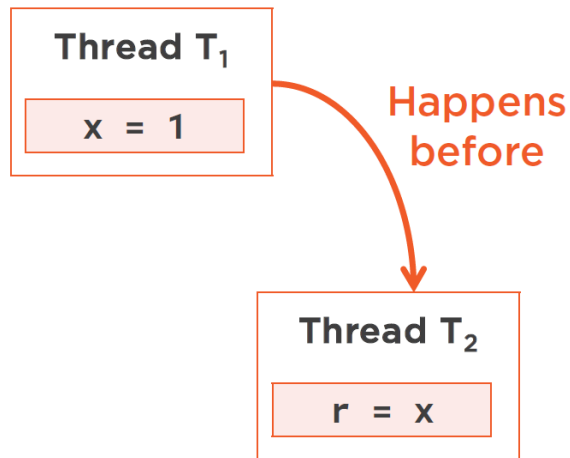
Variable is visible if writes made on it are visible. All synchronized writes are **VISIBLE**.

Happens Before Link

WE need a "**Happens Before LINK**" between READ and WRITE operations. JAVA '**Happens Before Link**' helps on write/read operations on multicore CPUs, e.g. BlockingQueue, CyclicBarrier, CountdownLatch etc.

Problems regarding multicore CPUs

- Read/Write can happen at same time (comparing to single core CPU)
- Access to cache is much more faster than accessing Main Memory
- Variable can be stored in several places (main memory, caches, ..)
- Visibility: Read operation should return always the latest value set by



If there is no “happens before” link between the two operations, the value of r is unknown

If there is a “happens before” link between the two operations, the value of r is 1

Example

- Thread1 write 1 to variable X.
- Thread2 read X and assign it to R
- What can be value of R, 0 (default initial value) or 1?
- Java Memory Model fixes it & return 1
- BUT how to tune JVM for **Happens Before Link**?

Happens Before Link **exists** between all

- Synchronized or **VOLATILE** write opts.
- Synchronized or VOLATILE read opts.

What Does Visibility Do? Guarantees the consistency of the variables (read always correct value)

All shared variables should be accessed in a synchronized or a volatile way, otherwise your will end up with race condition.

Happens Before Link Examples

What is the value of r2, if code is run by T1 and T2?

```
int x, y, r1, r2;
Object lock = new Object();
void firstMethod() {
    x = 1;
    synchronized(lock) {
        y = 1;
    }
}
void secondMethod() {
    synchronized(lock) {
        r1 = y;
    }
    r2 = x;
}
```

If T_1 is the first to enter the synchronized block, then the execution is in this order:

- $x = 1$
 - $y = 1$
 - $r1 = y$
 - $r2 = x$
- Happens-before link between a synchronized write and a synchronized read

The value of r2 is 1

If T_2 is the first to enter the synchronized block, then the execution is in this order:

- $r1 = y$
 - $r2 = x$ or $x = 1$?
 - $y = 1$
- No happens-before link between $r2 = x$ and $x = 1$

The value of r2 may be 0 or 1

What Does Visibility Do?

- Guarantees the consistency of the variables (read always correct value)

All shared variables should be accessed in a synchronized or a volatile way, otherwise you will end up with race condition.

Thread Safe Singleton Pattern - Double Check Lock

```
public static Singleton getInstance() {  
    if (instance != null) {  
        return instance;  
    }  
  
    synchronized(key) {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Looks like solve our READING problem, ... BUT
Introduces a BUG (**Visibility Issue**)

Use double check lock mechanism to make READ OPERATION available for parallel read.

- Is it synchronized or volatile WRITE? YES
- Is it synchronized or volatile READ? NO

So we have no guarantee that, READ will get value set by WRITE.

It is guaranteed only by “Happens Before Link” between READ/WRITE.

Best is using Java Concurrency API - Exclusive **ReadWriteLock** object, which allows lockWrite (just one thread can use), and lockRead(which allows parallel READ for multiple threads)

Thread Safe Singleton Pattern - Double Check Lock Fixed

```
public class Singleton {  
  
    private static volatile Singleton instance;  
  
    private final Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance != null) {  
            return instance;  
        }  
  
        synchronized(key) {  
            if (instance == null) {  
                instance = new Singleton();  
            }  
            return instance;  
        }  
    }  
}
```

Problem with double-checking comes from non-synchronized READ. So, solution is simple, use VOLATILE Read.

BUT we end up with Performance PENALTY again like a solution in “Synchronized Pattern”

Thread Safe Singleton Pattern – Enum Pattern

```
public enum Singleton {  
  
    INSTANCE  
}
```

Best solution is using ENUM Pattern. It provides full thread safety solution and free **serialization**

+ Very easy to implement

+ Gives best performance and no READ issue (**Visibility**)

+ No problem with reflection

e.g. Java 8 Comparator#naturalOrder()

See `SingletonSerializable.java` and `SingletonSerializableEnum.java` for **Serialization DEMO**

See `SingletonWithReflection.java` for **Reflection DEMO**

Singleton is Really Anti pattern?

While the Singleton Design Pattern has several advantages, particularly in terms of thread safety and lazy initialization, there are certain [drawbacks](#) to consider

- 1.Global State:** makes it harder to track and reason about the behavior of the system, e.g. changes to the Singleton instance can affect other application parts, potentially resulting in hidden bugs and unintended side effects. **The main problem is that the global variables are visible to all threads in our code**
- 2.Tight Coupling:** use of Singleton can lead to tight coupling between classes, e.g. make it more difficult to modify or replace Singleton
- 3.Testing Challenges:** Singleton instance is globally accessible, it can be challenging to isolate and test dependent components. Mocking or substituting the Singleton instance during testing may require additional work or codebase changes.
- 4.Difficulty in Lifecycle Management:** Singletons have a long lifetime in application. Managing the lifecycle of a Singleton, such as releasing resources or resetting the state, can be complicated, particularly in cases where the instance must be re-initialized.
- 5.Limited Extensibility:** Due to their restricted instantiation and global access, it can be difficult to extend or modify Singleton classes. Introducing new variations or behaviors to the Singleton without modifying the existing implementation can be challenging, too.
- 6.Violates the Single Responsibility Principle:** This principle states that every class should have a single task to do. In case of Singleton class, it will have two responsibility one to create an instance and other to do the actual task.
- 7.Breaks the Open Closed Principle:** The Singleton class always returns its own instance and is never open for extension.

Some are, not fair criticisms ;) ;)

Some known problems with singletons

Bijection violation Singletons don't exist in the real world.

Tight coupling They provide a hard-to-break global point of access.

Accidental implementation It is too related to implementation and does not mimic real-world behavior.

Hard testability It is hard to write unit tests when singletons are present.

Does not save memory Modern garbage collectors deal better with volatile objects than a permanent one.

Breaks dependency injection It is more difficult to decouple dependencies.

Instantiation contract violation When you ask a class to create an instance, you expect a new one.

Fail fast violation You shouldn't be able to create new instances, and they should fail instead of giving you an old instance.

Implementation coupling You use getInstance() instead of the new() method.

More difficult to apply the testdriven development (TDD) technique

TDD deals with coupling and you need to circumvent it when creating new tests.

Unique concepts are contextual See the previous code example. Uniqueness depends on the scope. And it never should be global.

Multithreaded environment problems

Many singletons are neither thread-safe nor reentrant and cause unexpected behavior.

Accumulate garbage state Running multiple tests might bloat the singleton, and since it is not collected, the garbage remains.

Class single responsibility violation / separation of concerns

Class's single responsibility is to create instances, not to manage them.

Easy friend for entry point Once you have a singleton, you can start polluting this convenient global reference with more objects.

Dependency hell A class depends on a singleton object, and that singleton object depends on another singleton object, and so on.

Lack of flexibility Once a singleton object is created, it cannot be replaced or modified.

Difficulty in lifecycle management Managing the lifecycle of a singleton can be challenging and may lead to memory leaks or unnecessary resource utilization.

When should we use the Singleton Design Pattern?

Single Instance Requirement: Only one instance needed in entire app., ... multiple instances would lead to conflicts

Global Access: We want a single point of access to the instance, allowing other parts of the application to easily use and interact with it. This eliminates the need for passing instances between objects or managing complex dependencies.

Limited Resource Usage: The class represents a limited or expensive resource, such as a database connection, file system, or hardware device, and we want to share that resource efficiently among multiple parts of the application.

Coordination and Control: We need a centralized point of coordination or control, where changes made to the instance can be observed or managed by other parts of the application.

Real-world examples, in practice

1.Logger: It ensures that only one instance of the Logger exists, and all parts of the system can access it easily.

2.Database Connection: A single, shared connection that multiple components can utilize.

3.Configuration Manager (centralized configuration to store settings and properties): Singleton ConfigurationManager class loads and manages the application configuration. This ensures that all parts of the application access the same configuration data.

4.Cache Manager: Caching is used to improve performance by storing frequently accessed data in memory. Singleton CacheManager class ensuring that only one cache instance exists and is accessible from multiple components.

5.Thread Pool: In concurrent programming, a thread pool is a common design pattern where a pool of threads is created to execute tasks. A Singleton ThreadPool class manages the creation, allocation, and execution of threads, providing a shared resource for handling concurrent tasks

6.Spring Singleton (singleton – one bean per identifier, easy testable, easy mockable, ...)

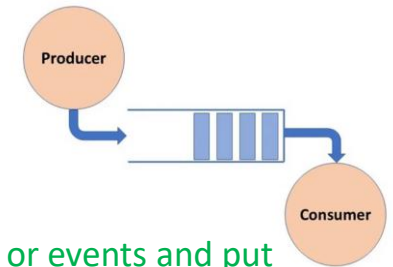
7.java.awt.Desktop#getDesktop(), java.lang.Runtime#getRuntime(), **Java Servlets** ;) [not keep state] (HttpServlet is a singleton) , Logger, . .

Producer / Consumer (or publisher-subscriber) pattern

Producer – produces values in a buffer (non full), whereas **Consumer** consumes those values from the buffer (non empty). Both run on its own thread, so consider **race condition** on shared buffer unless it is properly **synchronized**.

```
Int count=0;
int[] buffer = new int[BUFFER_SIZE];
Class Producer {
    public void produce() {
        while (isFull(buffer)) { //tbd
        }
        buffer[count++] = 1;
    }
}
```

```
class Consumer {
    public void consume() {
        while (isEmpty(buffer)) { //tbd
        }
        buffer[--count] = 0;
    }
}
```



In the Producer-Consumer pattern, producers generate data or events and put them into a shared data store or message queue.

What is wrong with above code?

Once producer and consumer run by multiple threads race condition will happen regarding the shared buffer.

1-try. What happens if we add **synchronized** keyword to above produce() and consume() methods?

Will it fix the problem? No, because consumer() and produce() methods use different LOCKs, reason is each class has its own instance bounded to synchronized method or synchronized block.

Just remember from above section we know that there are 3-ways of designating LOCK object.

2-try. Using **same lock object** for both consumer and producer fix the problem, but what if buffer is **empty or full**?

3-try. Use **wait/notify pattern** (must be in **synchronized block**) to improve the solution mentioned by 2-try

Producer / Consumer pattern



```
public class ProducerConsumerDemo {  
    /**  
     * Producer can not publish if buffer is full and consumer can not consume if it is empty.  
     * So, always keep buffer under control to prevent race condition.  
     */  
    private static int[] buffer;  
    private static int count;  
  
    private static Object monitor = new Object();  
  
    static class Producer {  
        public void produce() {  
            synchronized (monitor) {  
                if (isFull(buffer)) {  
                    try {  
                        monitor.wait(); //release KEY, will be available for consumer  
                        Thread.currentThread() = Thread.currentThread();  
                        System.out.println(currentThread.getName() + " state: " + currentThread.getState());  
                    } catch (InterruptedException e) {  
                        System.err.println(e.getMessage());  
                    }  
                }  
                buffer[count++] = 1; //without synchronization causes race condition  
                monitor.notifyAll();  
            }  
        }  
    }  
  
    static class Consumer {  
        public void consume() {  
            synchronized (monitor) {  
                if (isEmpty(buffer)) {  
                    try {  
                        monitor.wait();  
                        Thread.currentThread() = Thread.currentThread();  
                        System.out.println(currentThread.getName() + " state: " + currentThread.getState());  
                    } catch (InterruptedException e) {  
                        System.err.println(e.getMessage());  
                    }  
                }  
                buffer[--count] = 0; //without synchronization causes race condition  
                monitor.notifyAll();  
            }  
        }  
    }  
}
```

Producer Consumer Pattern

Wait-Notify Pattern

Classical way to implement this pattern is using wait()/notify() methods - see Part 1.

Problem with this approach is **we can not interrupt a waiting thread**, only way is to reboot JVM.

```
Lock lock = new ReentrantLock();
Condition notFull = lock.newCondition();
Condition notEmpty = lock.newCondition();
```

```
class Producer {
    public void produce() {
        try {
            lock.lock();
            while (isFull(buffer))
                notFull.await();
            buffer[count++] = 1;
            notEmpty.signal();
        } finally {
            lock.unlock();
        }
    }
}
```

```
class Consumer {
    public void consume() {
        try {
            lock.lock();
            while (isEmpty(buffer))
                notEmpty.await();
            buffer[--count] = 0;
            notFull.signal();
        } finally {
            lock.unlock();
        }
    }
}
```

```
Object lock = new Object();
```

```
class Producer {
    public void produce() {
        synchronized(lock) {
            while (isFull(buffer))
                lock.wait();
            buffer[count++] = 1;
            lock.notifyAll();
        }
    }
}
```

```
class Consumer {
    public void consume() {
        synchronized(lock) {
            while (isEmpty(buffer))
                lock.wait();
            buffer[--count] = 0;
            lock.notifyAll();
        }
    }
}
```

We use explicit **Lock** to implement this pattern to fix above issue. Offer controlled interrupt ability, timeout, and fairness.

Condition await() method is equivalent with wait() method
Condition signal()/signalAll() method is equivalent with notify()/notifyAll() method in classical approach

(see Condition class in previous section)

